

Домашнее задание №2: Индексы и SQL-функции

Тема: ускорение поиска, CREATE INDEX, EXPLAIN, пользовательские функции PostgreSQL

Цель

Научиться понимать, зачем нужны индексы, где их ставить, как проверять запрос через EXPLAIN, а также писать простые SQL-функции для повторяемой логики.

Что сдать

Один файл homework_indexes_functions.sql. Внутри должны быть: запросы для проверки, CREATE INDEX, EXPLAIN/EXPLAIN ANALYZE, функции и тестовые SELECT-вызовы функций.

База данных

Используйте ту же схему книжного магазина. Если таблицы уже созданы, повторно CREATE TABLE выполнять не нужно.

```
CREATE TABLE categories(  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE  
);  
  
CREATE TABLE books(  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(150) NOT NULL,  
  price NUMERIC(10,2) NOT NULL,  
  stock INT NOT NULL DEFAULT 0,  
  category_id INT REFERENCES categories(id)  
);  
  
CREATE TABLE authors(  
  id SERIAL PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE book_authors(  
  book_id INT REFERENCES books(id),  
  author_id INT REFERENCES authors(id),  
  PRIMARY KEY (book_id, author_id)  
);  
  
CREATE TABLE customers(  
  id SERIAL PRIMARY KEY,  
  full_name VARCHAR(100) NOT NULL,  
  phone VARCHAR(20),  
  email VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE orders(  
  id SERIAL PRIMARY KEY,  
  customers_id INT REFERENCES customers(id),  
  order_date DATE NOT NULL DEFAULT CURRENT_DATE  
);  
  
CREATE TABLE orders_items(  
  id SERIAL PRIMARY KEY,  
  order_id INT REFERENCES orders(id),  
  book_id INT REFERENCES books(id),  
  quantity INT NOT NULL,  
  price_at_moment NUMERIC(10,2) NOT NULL  
);
```

Минимальные данные для проверки: 5 категорий, 10 книг, 6 авторов, 8 клиентов, 10 заказов, 18 позиций заказов. Данные можно взять из урока или создать самостоятельно через INSERT.

Часть А. Индексы

1. Написать запрос, который ищет книгу по точному названию: WHERE title = ... Затем создать индекс для ускорения такого поиска.
2. Написать запрос, который выводит книги конкретной категории через WHERE category_id = ... Создать индекс на внешний ключ category_id.
3. Написать запрос, который ищет заказы за конкретный период по order_date. Создать индекс на order_date.
4. Написать запрос, который ищет клиента по email. Проверить, есть ли индекс уже автоматически из-за UNIQUE. Объяснить комментарием в SQL-файле.
5. Создать составной индекс для частого отчета по позициям заказа: order_id + book_id. Написать запрос, где такой индекс может быть полезен.
6. Создать индекс для связи book_authors по author_id, чтобы быстрее находить все книги конкретного автора.

Часть В. Проверка через EXPLAIN

7. Для задания 1 показать EXPLAIN до создания индекса и после создания индекса. Добавить оба варианта в файл как комментарии или отдельные запросы.
8. Для задания 3 показать EXPLAIN ANALYZE запроса по диапазону дат. Коротко написать комментарием, что изменилось после индекса.
9. Написать один пример, где индекс почти не помогает. Например, запрос выбирает слишком много строк или использует условие, которое плохо подходит под индекс. Объяснить одной строкой комментария.

Часть С. Встроенные SQL-функции

10. Вывести книги в формате: название книги большими буквами, цена, цена со скидкой 10%. Использовать UPPER и арифметику.
11. Вывести клиентов: ФИО, email, длина email. Использовать LENGTH.
12. Вывести заказы: id заказа, дата заказа, год заказа, месяц заказа. Использовать EXTRACT.
13. Вывести позиции заказов: название книги, quantity, price_at_moment, итоговая сумма позиции. Итоговую сумму округлить до 2 знаков через ROUND.

Часть D. Пользовательские функции PostgreSQL

```
Пример формы функции:  
CREATE OR REPLACE FUNCTION function_name(param TYPE)  
RETURNS TYPE AS $$  
    SELECT ...;  
$$ LANGUAGE sql;
```

14. Создать функцию get_discount_price(book_price NUMERIC, discount_percent NUMERIC), которая возвращает цену со скидкой.
15. Создать функцию get_order_total(order_id_param INT), которая возвращает итоговую сумму заказа по таблице orders_items.
16. Создать функцию get_books_count_by_category(category_id_param INT), которая возвращает количество книг в категории.
17. Создать функцию get_customer_orders_count(customer_id_param INT), которая возвращает количество заказов клиента.

18. Создать функцию `get_book_sold_quantity(book_id_param INT)`, которая возвращает суммарное количество проданных экземпляров книги. Если продаж нет, вернуть 0 через `COALESCE`.
19. Для каждой функции написать минимум 2 тестовых `SELECT`-вызова.
20. В конце файла написать короткий комментарий: какие 2 индекса в этой базе самые полезные и почему.

Требования к выполнению

- Все индексы создавать через `CREATE INDEX` или `CREATE INDEX IF NOT EXISTS`.
- Имена индексов должны быть понятными: `idx_books_title`, `idx_orders_order_date` и т.д.
- Не создавать индекс на каждую колонку без смысла. Индекс нужен под конкретный запрос.
- Функции должны быть рабочими и возвращать один результат.
- После каждой функции должны быть тестовые `SELECT`-запросы.

Критерии оценивания: 12 баллов

Критерий	Баллы
Корректно созданы индексы под реальные запросы.	3
Есть проверка <code>EXPLAIN / EXPLAIN ANALYZE</code> и короткие выводы в комментариях.	2
Правильно использованы встроенные функции: <code>UPPER</code> , <code>LENGTH</code> , <code>EXTRACT</code> , <code>ROUND</code> .	2
Пользовательские функции написаны корректно и возвращают нужные значения.	3
Есть тестовые <code>SELECT</code> -вызовы функций и аккуратное оформление файла.	2